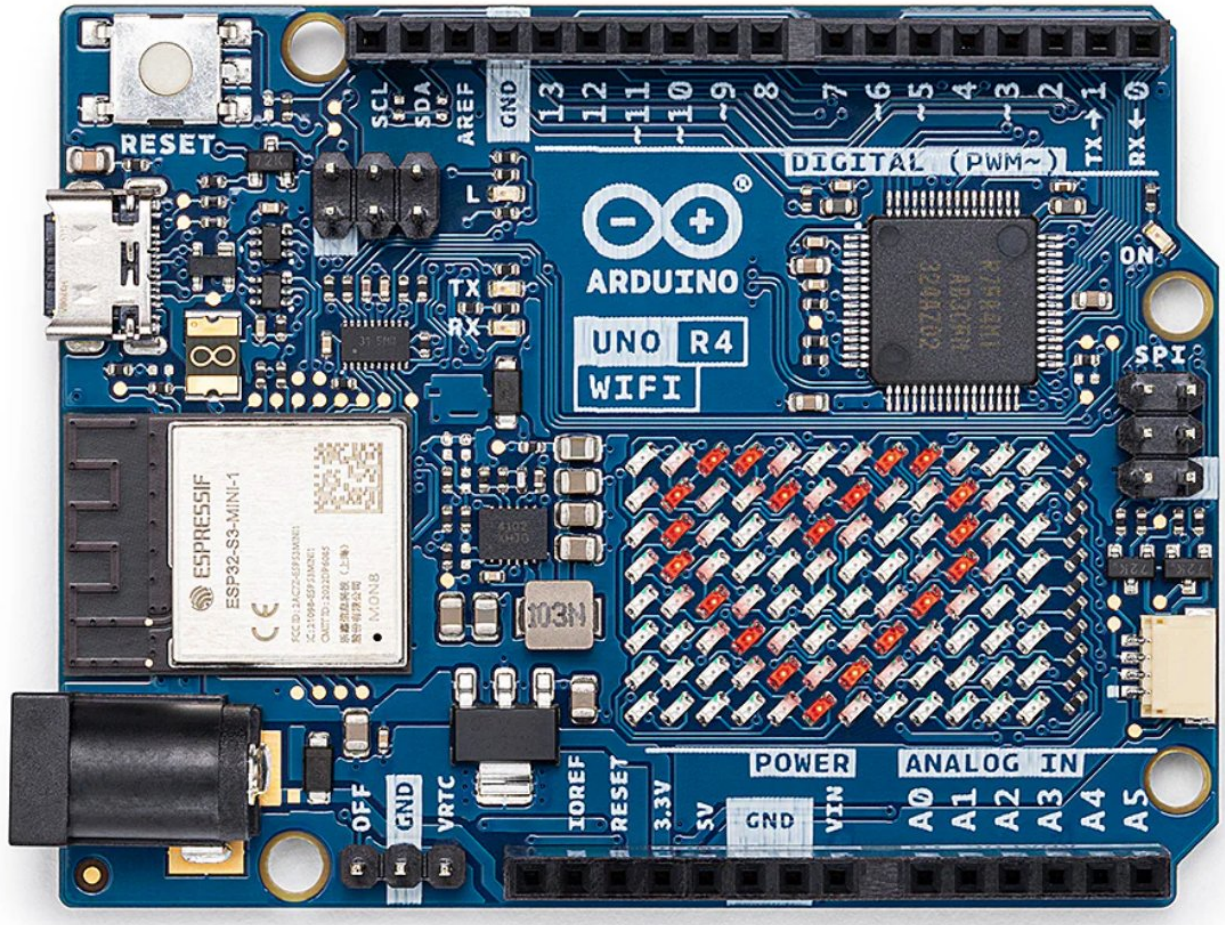
Arduino Plug & Make Kit — Phase 3

Live Sensor Web Page

WiFiS3 · Modulino Thermo · Arduino as a Web Server



What you will build	What you will learn
The Arduino connects to your WiFi and hosts its own web page	How WiFi works, what an IP address is, how web servers work
Open any browser on the same WiFi and type the Arduino's address	How a browser asks for a page and how a server sends one back
The page shows live temperature and humidity from the Thermo module	How sensor data can be sent over a network in real time
The page refreshes automatically every 5 seconds	How to include HTML inside Arduino code

No app. No cloud account. No extra software. Just your Arduino, your WiFi, and a browser.

How This Project Works

When you visit a website, your browser sends a request to a **web server** — a computer listening for those requests and ready to reply with a web page. Normally a web server is a powerful machine in a data centre somewhere. In this project, **your Arduino is the web server**.

Every device on a WiFi network has a unique address called an **IP address** — four numbers separated by dots, like **192.168.1.47**. When the Arduino connects to your WiFi it gets its own IP address. You type that address into your browser and the Arduino sends back a page showing the current temperature and humidity.

The page uses a built-in HTML trick called **meta refresh** to reload itself every 5 seconds, so the readings update automatically.

Step	What happens
1 — Arduino boots	Connects to your WiFi and gets an IP address
2 — Arduino starts server	Begins listening on port 80 for browser requests
3 — You open a browser	Type the Arduino's IP address into the address bar
4 — Browser sends request	Asks the Arduino for its web page
5 — Arduino reads sensor	Gets current temperature and humidity from Thermo
6 — Arduino sends page	Builds an HTML page with the live values and sends it
7 — Browser shows page	You see the readings. Page reloads every 5 seconds

> **Tip:** Both your Arduino and your computer (or phone) must be on the **same WiFi network**. The Arduino cannot be reached from outside your home.

What You Need

Item	Notes
Arduino UNO R4 WiFi	WiFiS3 library is already included — nothing extra to install
Modulino Thermo module	Provides temperature and humidity readings
Qwiic cable	Connects the Thermo module to the Arduino
USB-C cable	To upload the sketch and power the board
Your WiFi name and password	2.4 GHz networks only — R4 WiFi does not support 5 GHz
A phone or computer on the same WiFi	Any device with a browser works

Important: The UNO R4 WiFi only connects to **2.4 GHz** networks. Many modern routers broadcast both 2.4 GHz and 5 GHz — sometimes with the same name. If the Arduino fails to connect, check your router and make sure you are using the 2.4 GHz band.

Wiring

This project needs just one Qwiic cable. No breadboard, no loose wires.

From	To	Notes
Modulino Thermo — any Qwiic port	Arduino UNO R4 WiFi — Qwiic connector	Either port on the Thermo works — they are identical

Tip: The Qwiic connector on the Arduino is the small gold connector on the right side of the board. The cable only fits one way — do not force it.

The Code

Before uploading, find the two lines near the top that say **YOUR_WIFI_NAME** and **YOUR_WIFI_PASSWORD** and replace them with your actual WiFi details. Keep the quotation marks — only replace the words inside them.

> **Important:** WiFi passwords are case-sensitive. One wrong capital letter will stop the Arduino connecting.

```
// ============================================================
// Live Sensor Web Page
// Arduino UNO R4 WiFi + Modulino Thermo
// The Arduino hosts a web page on your WiFi showing live
// temperature and humidity from the Thermo module.
// Open a browser on the same WiFi, type in the IP address
// shown in the Serial Monitor, and see the live readings.
// ============================================================
#include <WiFiS3.h> // The WiFi library — built into the UNO R4 board package
#include <Modulino.h> // The Modulino library for all sensor modules
// ---- Enter your details here ----
const char ssid[] = "YOUR_WIFI_NAME"; // Replace with your WiFi network name
const char pass[] = "YOUR_WIFI_PASSWORD"; // Replace with your WiFi password
const char station[] = "YOUR_STATION_NAME"; // Replace with your weather station name
ModulinoThermo thermo; // Creates a thermo object so we can talk to the sensor
WiFiServer server(80); // Creates a web server listening on port 80
// Port 80 is the standard port for web pages (HTTP)
// ============================================================
// setup() runs once when the Arduino is switched on
// ============================================================
void setup() {
Serial.begin(9600); // Start Serial Monitor — used to show the IP address
Modulino.begin(); // Start the Modulino system so sensors can be used
thermo.begin(); // Wake up the Thermo sensor
// Connect to WiFi
Serial.print("Connecting to WiFi");
WiFi.setHostname(station); // Give this board a name on the network
WiFi.begin(ssid, pass); // Tell the Arduino to connect using our details
// Keep looping here until the WiFi connects
// WiFi.status() checks the connection — WL_CONNECTED means success
while (WiFi.status() != WL_CONNECTED) {
delay(500);
Serial.print("."); // Print a dot each attempt so we can see progress
}
Serial.println("");
Serial.println("WiFi connected!");
// Wait until the board has been assigned a real IP address
// 0.0.0.0 means the router has not given us an address yet
while (WiFi.localIP() == IPAddress(0, 0, 0, 0)) {
delay(500);
Serial.print(".");
}
Serial.println("");
```

```
Serial.print("Open your browser and go to: http://");
Serial.println(WiFi.localIP()); // This is the address to type in your browser
server.begin(); // Start the web server — ready to receive browser requests
}
// ============================================================
// loop() runs over and over forever
// It checks if a browser has connected and sends back a page
// ============================================================
void loop() {
// Check if a browser has connected
// server.available() returns a client if someone connected, otherwise nothing
WiFiClient client = server.available();
if (client) { // A browser has connected — time to send it a web page
// Read and discard the browser's request
// Every browser sends a block of text asking for the page
// We do not need to read it — we always send the same page back
String request = "";
while (client.connected()) {
if (client.available()) {
char c = client.read();
request += c;
if (request.endsWith("\r\n\r\n")) break; // Blank line = end of request
}
}
// Read fresh sensor values right now
float temperature = thermo.getTemperature(); // Degrees Celsius
float humidity = thermo.getHumidity(); // Percentage 0-100
// ---- HTTP response header ----
// This must be sent before any HTML
// It tells the browser what kind of content is coming
client.println("HTTP/1.1 200 OK"); // 200 OK means the request worked
client.println("Content-Type: text/html"); // Tells browser to expect HTML
client.println("Connection: close"); // Close connection after sending
client.println(); // Blank line = end of headers
// ---- HTML page starts here ----
// HTML is the language all web pages are written in
// Every tag that opens must be closed — <h1> opens, </h1> closes
client.println("<!DOCTYPE html>"); // Tells the browser this is HTML5
client.println("<html><head>"); // Start of the page, then the head section
// The head contains settings — not visible content
// The viewport tag makes the page fit correctly on phones and tablets
// Without it, mobile browsers zoom out and everything looks tiny
client.println("<meta name='viewport' content='width=device-width, initial-scale=1'>");
```

```
// The title appears in the browser tab at the top of the screen
// We use the station name variable so it updates automatically
client.print("<title>");
client.print(station);
client.println("</title>");
// Meta refresh tells the browser to reload the page every 5 seconds
// This is how the readings stay live — the page keeps fetching new data
// Change the number 5 to reload faster or slower
client.println("<meta http-equiv='refresh' content='5'>");
// ---- CSS styles ----
// CSS controls how everything looks — colours, sizes, spacing
// Styles are written between <style> and </style> tags in the head
client.println("<style>");
// body — styles the whole page
// font-family: the font to use
// background: the page background colour (#f5f5f5 is light grey)
// text-align: center puts everything in the middle
// padding: adds space around the edges so content is not right against the screen
client.println("body { font-family: Arial, sans-serif; background: #f5f5f5;");
client.println(" text-align: center; padding: 40px; color: #1C2833; }");
// h1 — styles the main heading
// #005C5F is a dark teal colour
client.println("h1 { color: #005C5F; }");
// .box — styles each reading card
// A class name starting with . applies to any element with that class
// border-radius rounds the corners
// border adds the teal outline
// max-width stops the box getting too wide on large screens
client.println(".box { background: white; border-radius: 12px;");
client.println(" border: 2px solid #00979D; padding: 24px;");
client.println(" margin: 20px auto; max-width: 340px; }");
// .val — styles the big number inside each box
// font-size: 3em means 3 times the normal text size
client.println(".val { font-size: 3em; color: #00979D; font-weight: bold; }");
// .lbl — styles the small label below each number (Temperature, Humidity)
client.println(".lbl { font-size: 1em; color: #888; margin-top: 6px; }");
// .note — styles the small grey text at the bottom
client.println(".note { font-size: 0.8em; color: #aaa; margin-top: 28px; }");
client.println("</style>"); // End of CSS styles
client.println("</head>"); // End of the head section
client.println("<body>"); // Start of the body — this is the visible page content
// The main heading — uses the station name variable
client.print("<h1>");
```

```
client.print(station);
client.println("</h1>");
// ---- Temperature card ----
// <div class='box'> creates a box using the .box style we defined above
client.println("<div class='box'>");
// <div class='val'> makes the number big and teal
client.print(" <div class='val'>");
client.print(temperature, 1); // Print temperature to 1 decimal place
client.println(" &deg;C</div>"); // &deg; is the HTML code for the degree symbol °
// <div class='lbl'> adds the small label underneath
client.println(" <div class='lbl'>Temperature</div>");
client.println("</div>"); // Close the box div
// ---- Humidity card ----
// Same structure as the temperature card
client.println("<div class='box'>");
client.print(" <div class='val'>");
client.print(humidity, 1); // Print humidity to 1 decimal place
client.println(" %</div>"); // % is the unit for humidity
client.println(" <div class='lbl'>Humidity</div>");
client.println("</div>");
// ---- Footer notes ----
// <p> is a paragraph tag — used for small blocks of text
client.println("<p class='note'>Updates every 5 seconds</p>");
// Show the IP address on the page so you can find it again easily
client.println("<p class='note'>");
client.println(WiFi.localIP());
client.println("</p>");
client.println("</body></html>"); // Close the body and html tags — page is complete
client.stop(); // Disconnect — the browser will reconnect automatically on the next refresh
}
}
```

What to Expect

1 Upload the sketch

Replace YOUR_WIFI_NAME and YOUR_WIFI_PASSWORD with your real details, then click the Upload arrow in the IDE.

2 Open the Serial Monitor

Click **Tools** → **Serial Monitor**. Set the baud rate to **9600**. You will see dots while the Arduino tries to connect — usually 5–15 seconds. Once connected you will see:

```
WiFi connected!

Open your browser and go to: http://192.168.X.X
```

The numbers at the end will be different on your network — that is your Arduino's IP address.

3 Open the web page

On any device on the same WiFi, open a browser and type the IP address exactly as shown in the Serial Monitor. Press Enter.

You will see a white page with two teal boxes showing the live temperature and humidity. The page updates every 5 seconds automatically.

Success: The web page is live and showing real sensor readings. Try breathing on the Thermo module — watch the humidity rise within seconds.

Troubleshooting

Problem	Likely cause	What to try
Dots in Serial Monitor forever — never connects	Wrong WiFi details or 5 GHz network	Check ssid and pass carefully. Confirm it is a 2.4 GHz network. Press Reset on the board.
IP shown but browser says page not found	Device is on a different network	Make sure your phone/laptop is on the same WiFi. Turn off mobile data on phones.
Page loads once but stops updating	Connection dropped	Refresh manually. If still failing, press Reset on the Arduino.
Temperature shows 0.00	Thermo not detected	Check the Qwiic cable at both ends. Press Reset.
WiFiS3.h not found	Board package missing	Go to Tools → Boards Manager and install Arduino UNO R4 Boards.

Things to Try Once It Is Working

- **Change the refresh rate** — find content='5' in the HTML and change 5 to 2. The page now updates every 2 seconds.

- **Add a temperature message** — after reading temperature, add an if statement: if it is above 25, print an extra line saying 'Too warm!'
- **Change the colour scheme** — find #00979D in the HTML style section and replace it with any hex colour. Try #E74C3C for red.
- **Show both readings in one line** — instead of two separate boxes, try combining them: 22.5 C / 58% humidity.
- **Add a page title with the IP address** — use WiFi.localIP() to include the board's address in the page heading.